Course Code: SL-3003	Course : Web Engineering Lab
Course Code: SL-3003	Course: Web Engineering Lab

Spring 2025, Lab Manual – 05

REACT JS

Topics to be studied

- **Introduction of REACT**
- **Installation of React**
- **Components (function, class)**
- **Introduction of JSX**
- **Events**
- **States**
- **Props**

Introduction of **React JS**

ReactJS is an open-source JavaScript library for building dynamic and interactive user interfaces (UIs). React is developed and released by Facebook. Facebook is continuously working on the React library and enhancing it by fixing bugs and introducing new features. It is popular due to its component-based architecture, Single Page Applications (SPAs) and Virtual DOM for building web applications that are fast, efficient, and scalable.

Code example

```
import React from 'react';
 import ReactDOM from 'react-dom/client';
  function Greeting(props) {
```

```
  return <h1>hello world </h1>;
}
 const container = document.getElementById("root");
 const root = ReactDOM.createRoot(container);
 root.render(<Greeting />);
```

Why Learn React JS?

React, the popular JavaScript library, offers several exciting reasons for developers to learn it.

- Flexibility: React allows you to build high-quality user interfaces across different platforms, thanks to its flexible library approach.
- Great Developer Experience: React makes it easier to write and understand code, offering a smooth development process.
- Strong Support from Facebook: Regular updates, bug fixes, and resources from Facebook ensure React stays up-to-date and reliable.
- Vast Community Support: React benefits from a large community, providing enough resources, tutorials, and troubleshooting help.
- Excellent Performance: React's efficient Virtual DOM ensures fast rendering and high performance for web applications.
- Easy Testing: React's structure and tools make testing simpler, improving code reliability and maintainability.

How to install React JS?

- Intall node.js
- Create a folder and open the directory on cmd
- **npx create-react-app** for installation of react
- **npm start** to run the react server
- **Set-ExecutionPolicy RemoteSigned** run by PowerShell administrator(if you any exception)

Create custom file

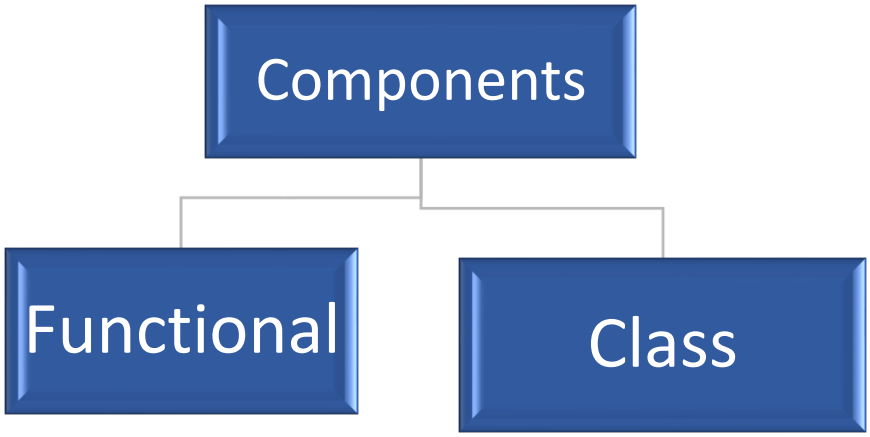
- All new files will be created into **src folder**

- Name of file should be capitalized e.g **Newfile.js**
- Copy the function from **app.js** file paste at **Newfile.js**
- Change the name of function with your file name
- Link at **index.js**

COMPONENTS

React components are independent, reusable building blocks in a React application that define what gets displayed on the UI. There are two primary types of React components

- **Functional Components**
- **Class Components**



Functional Components

Functional components are simpler and preferred for most use cases. They are JavaScript functions that return React elements.

Advantages

- Simpler Syntax: Ideal for small and reusable components.
- Performance: Generally faster since they don't require a 'this' keyword.

Source Code

Child file

```
// function FuncComponents(){ //for single component
export function FuncComponents(){ // for multiple components
return (
    <div>
        <h1>this is my first function component of react</h1>
    <h2>heading 2</h2>
    <h3>heading 3</h3>
```

```
    </div>

)
}
// export default FuncComponents; // for single component
```

Class Components

A class component requires you to extend from React. Component and create a render function that returns a React element.

Course code

```
import React from "react";
class ClassComponent extends React.Component {
    render(){
        return <h1>Class Component</h1>;
    }
}

export default ClassComponent;
```

Difference between function and class components

Functional Component	Class Component
A functional component is just a plain JavaScript pure function that accepts props as an argument and returns a React element(JSX).	A class component requires you to extend from React. Component and create a render function that returns a React element.
There is no render method used in functional components.	There is no render method used in functional components.
There is no render method used in functional components.	React lifecycle methods can be used inside class components (for example, componentDidMount).

Constructors are not used.	Constructor is used as it needs to store state.

Introduction of JSX

JSX stands for JavaScript XML. JSX is used to write the HTML-like code in JavaScript.JSX is basically a syntax extension of JavaScript. React JSX helps us to write HTML in JavaScript and forms the basis of React Development. Using JSX is not compulsory but it is highly recommended for programming in React as it makes the development process easier as the code becomes easy to write and read.

```
function JSX(){
    return (
        <p>
            this is JSX component (2 + 2) But using curly braces is
performance action of js {2+2}
            </p>
    )
}
export default JSX;
```

Practice question 1

A company wants to create an employee directory where each employee's details (name, position, and department) should be displayed inside a card. The company needs a React function component to take employee data as props and display it in a structured format.

Expected Output

Introduction of Events

React events are functions that get triggered in response to user interactions or other actions in the browser. They are similar to standard DOM events but have a consistent, cross-browser interface in React. React events use camelCase naming conventions and are attached to JSX elements as attributes.

- Using events like onChange or onInput to capture and process user input in text fields.
- Utilizing onClick to run functions when users click buttons or links.
- **Source Code**

```
function EventCom(){
    let firstName = "ali";
    let lastName = "mohamed";
    function UseEvent(){
        firstName ="sana";
        lastName = "khan";
        alert(firstName+" "+lastName);
    }
    console.log("rendering")
    return(
        <div>
            <h1>{firstName+" "+ lastName}</h1>
            <button onClick={()=>UseEvent()}>click me </button>
        </div>
    )
}
export default EventCom;
```

Introduction of state

When a functional component receives input and is rendered, React uses props and updates the virtual DOM to ensure the UI reflects the current state.

- **Virtual DOM:** When the component is rendered, React creates a virtual DOM tree that represents the current state of the application.
- **Re-rendering:** If the component's props or state change, React updates the virtual DOM tree accordingly and triggers the component to re-render.
- **useState:** returns an array that needs to be destructured. It returns two values: one is the current state value, and the other is a function to update the state.

Source code

```
import React, {useState} from "react";
function StateFunCom(){
    // let [userName,setNewName]=useState("asad");
    let [count,setCounter]=useState(0); //for task

    function FunUpdate(){
        // setNewName("yasir");
        setCounter(count+1);

    }
    console.log("rendering");
    return(
        <div>
            <h1>{count}</h1>
            <button onClick={FunUpdate}>click me</button>
        </div>
    )
}
export default StateFunCom;
```

Introduction of props

The react props refer to properties in react that are passed down from parent component to child to render the dynamic content.

Passing and Accessing props

We can pass props to any component as we declare attributes for any HTML tag.

- **Props:** Functional components receive input data through props, which are objects containing key-value pairs.

- **Processing Props:** After receiving props, the component processes them and returns a JSX element that defines the component's structure and content.

Source Code

Parent component

```
import logo from './logo.svg';
import './App.css'x`;
import PropsFunCom from './PropsFunCom';
function App() {
  return (
    <div className="App">
      <h1>this is my app file</h1>
    <PropsFunCom name = {"fareeha"} email =
{"f@gmail.com"} info =
{{salary:80000,no:364834648}}/>
    </div>
  );
}
export default App;
```

Source Code

Child component

```
function PropsFunCom(props){
    return (
        <div>
            <h1>Props and Fun Components and
the key is {props.name}</h1>
            <h2>Props and Fun Components and
the key is {props.email}</h2>
            <h2>Props and Fun Components and
the key is {props.info.salary}</h2>
            <h2>Props and Fun Components and
the key is {props.info.no}</h2>
        </div>
```

```
    )
}
export default PropsFunCom;
```

TASK 1

You are developing a Task Manager App where users can add predefined tasks, delete them, and mark them as complete without using input fields. Instead of typing a new task, users will click a button to add a predefined task from a list.

TASK 2

You need to create a dashboard layout with a sidebar menu on the left and a main content area on the right. The sidebar should contain links (dummy) like Dashboard, Profile, and Settings. When a user clicks a link, the corresponding content should be displayed in the main content area

Task 3

Create an e-commerce layout where a list of products is displayed. Clicking on a product should display its details on the same page.